

Procedimiento Estandarizado ante Errores

Suite de Auditoría y Sistema RAG

Prof. César Rodríguez

Curso: Seguridad Informática

19 de mayo de 2026

Introducción

En el desarrollo colaborativo de nuestra Suite de Auditoría, los errores son una oportunidad de aprendizaje grupal. Este documento describe los pasos obligatorios que cada estudiante debe seguir al encontrarse con fallos en el código (Python), problemas de red o errores de validación, asegurando que queden registrados en nuestra memoria colectiva (Sistema RAG) para su futuro análisis.

Paso 1: Captura del Error en el Historial

El orquestador principal (`auditoria.py`) está diseñado para registrar fallos en `historial_auditoria.json`. Para que esto funcione desde tu módulo (ej. `dns_recon.py`, `osint.py`):

- **Errores Controlados:** Si usas bloques `try-except`, asegúrate de atrapar la excepción y retornar el diccionario del contrato de datos con la clave `'status': 'error'` y el detalle del fallo en `'error_message'`.
- **Errores Inesperados:** Si tu script crashea repentinamente, el orquestador atrapará el fallo (Traceback) automáticamente y lo guardará en el JSON.

Paso 2: Exportar el Error a la Base de Conocimiento

Una vez que el error se ha guardado en `historial_auditoria.json`, debes extraerlo para que el sistema RAG (ChromaDB) pueda procesarlo. En tu terminal, ejecuta:

```
python exportar_errores_json.py
```

Este script leerá el historial y generará/actualizará el archivo `docs/auditoria.md`, creando una sección con el detalle del fallo y un espacio en blanco que dice “**La Solución (Pendiente)**”.

Paso 3: Investigación y Solución (Trabajo Colaborativo)

Con el error ya exportado a texto plano:

1. Abre el archivo `docs/auditoria.md` en tu editor.
2. Busca la sección del error reciente que causó tu módulo.
3. Investiga la solución con tu grupo. (Tip: Puedes consultar si un error similar ya ocurrió antes ejecutando `python knowledge_db.py -c auditoria -q 'tu problema'`).
4. Reemplaza la nota **(Agrega la solución aquí...)* con la explicación técnica de por qué ocurrió el fallo y cómo solucionarlo de manera definitiva.

Paso 4: Actualizar la Base Vectorial (RAG)

Para que el “cerebro” de la clase aprenda esta nueva solución de forma semántica, debes indexar el documento actualizado ejecutando:

```
python knowledge_db.py -c auditoria -u
```

A partir de este momento, cualquier estudiante que consulte el RAG con un problema similar encontrará tu solución.

Paso 5: Enviar los Cambios (Pull Request)

Finalmente, para que el profesor y el resto de los grupos reciban tanto tu corrección de código como la actualización de la documentación (`auditoria.md`):

```
git add .
git commit -m 'Fix: Solucionado error en modulo X y actualizado RAG de auditoria'
git push origin <nombre-de-tu-rama>
```

Abre tu *Pull Request* en GitHub e indica en la descripción que el error fue resuelto y documentado en el Sistema RAG.